



COURSE

HIGHER DIPLOMA IN SCIENCE IN COMPUTING (SOFTWARE
DEVELOPMENT)

MODULE

B8IT150 ADVANCED PROGRAMMING (B8IT150_2425_TME4)

MODULE LEADER

Kanza Ali Manzar

TOPIC

CA – ONE

SUBMITTED BY

Vinicius Quintanilha Sano – 20042134

Date: 15/12/2025

Contents

1. Project's Background	1
A) Purpose.....	1
B) Objective	1
C) Project's requirements.....	1
➤ Store, access and modify guest information.	1
➤ Store, access, modify and remove reservations.....	1
➤ Store and access room's details.	1
2. UML Class Diagram	2
Relationship between classes	2
Class methods	2
3. Entity Relationship Diagram (ERD)	4
Software Features and Functionalities	5
3.1. Presentation Layer (UI Layer).....	5
3.2. Business Logic Layer (BLL)	7
3.2.1. Classes.....	7
3.2.2. Object oriented programming concepts	8
3.2.3. Form validation rules	8
3.2.4. Services: Sorting and Searching.....	9
3.2.5. Methods and Logic.....	10
4.3. Data Access Layer (DAL)	16
4.3.1. Methods and Logic.....	16
5. Future Development.....	18
References	19

1. Project's Background

A) Purpose

To provide hotels with an efficient system for managing, tracking, and organizing guest reservations.

B) Objective

Develop a hotel reservation management system capable of storing, accessing, updating, and deleting guest reservations.

C) Project's requirements

➤ Store, access and modify guest information.

The software must be able to store guest's name, email address, phone number and address.

➤ Store, access, modify and remove reservations.

The software must be able to store reservation check-in and check-out dates, room type and number, price and the total number of guests staying in the room.

➤ Store and access room's details.

The software must be able to store room type (single room, double room, triple room, family room), room number, and maximum capacity.

2. UML Class Diagram

The UML class diagram was created to represent the static structure of the system, and is shown in Figure 1.

The diagram includes three main classes - **Guest**, **Reservation** and **Room** – and four subclasses (child) derived from the Room superclass (parent): Single Room, Double Room, Triple Room, and Family Room.

Relationship between classes

The relationship between **Guest** and **Reservation** is one-to-one, meaning that each **Guest** object is associated with only one **Reservation**, and each **Reservation** object is associated to only one **Guest**.

The relationship between **Room** and **Reservation** is one-to-many, meaning that each **Room** object is associate with one or more **Reservation** objects, but each **Reservation** object is associated with only one **Room**.

The relationship between **Room** and its subclasses characterize an Inheritance relationship. This means that the child classes inherit the attributes of the parent class and may also define their own specific attributes.

Class methods

A **Guest** object can be created using the **AddGuest()** method, which takes the guest's name, email, phone number and address as input parameters. Guest information can also be modified using the **UpdateGuest()** method.

A **Reservation** object can be create using the **AddReservation()** method, which takes the guests ID, room number, check-in and check-out dates, total number of guests, and total price as input parameters. Reservation details can be updated using the **UpdateReservation()** method and removed using the **DeleteReservation()** method, which requires the reservation ID as a parameter.

The **GetAvailableRoomsByTypeAndDate()** method in the **Room** class retrieves rooms by type, using the **GetRoomType()** method, and filters them by the specified check-in and check-out dates. The purpose of this method is to return only rooms that are available during the requested reservation period.

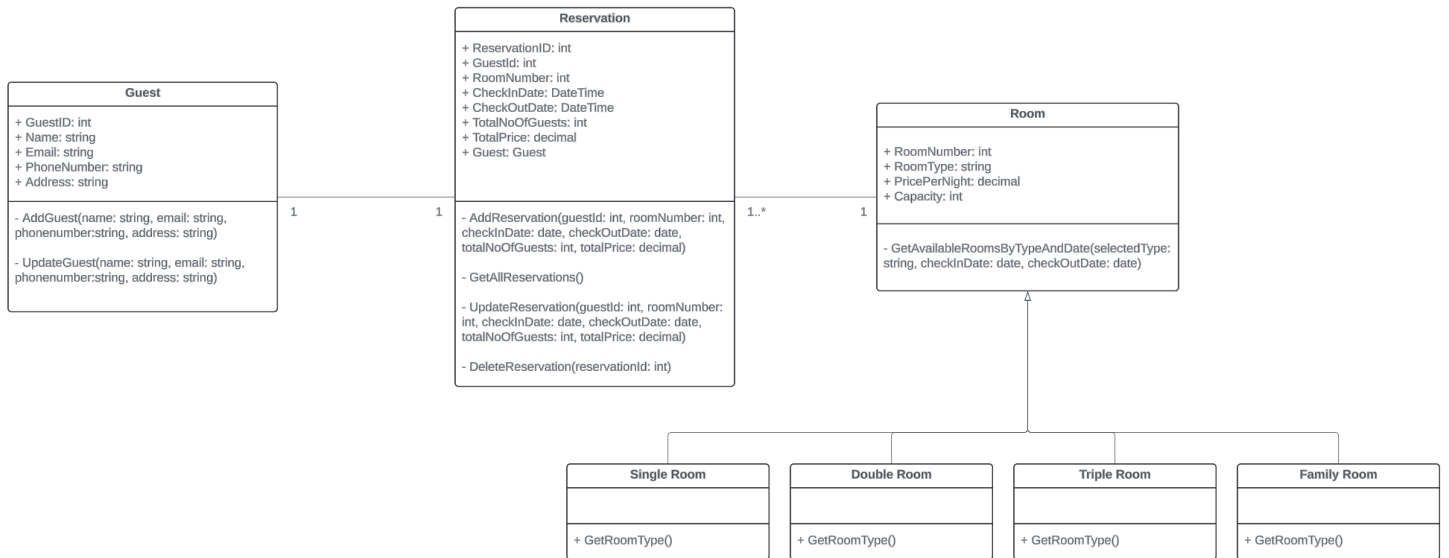


Figure 1 - UML class diagram from the Hotel Reservation Management System.

3. Entity Relationship Diagram (ERD)

The ERD diagram was created to represent the database structure and the relationships between entities (tables) and is shown in Figure 2.

The **Users** entity stores user credentials required to login in to the system. It does not have any relationship with other entities.

The **Guests** entity has a one-to-one relationship with the **Reservations** entity, meaning that each guest is associated with one, and only one, reservation. It also applies to the **Reservations** entity in relation to **Guests**. A reservation must have one, and only one, guest.

The **Reservations** entity has a one-to-one relationship with **Rooms** entity, meaning that each reservation is associated with one, and only one, room. However, the **Rooms** entity has a zero-to-many relationship with **Reservations**, meaning that a room may not be allocated to any reservation or may be allocated to multiple reservations over time.

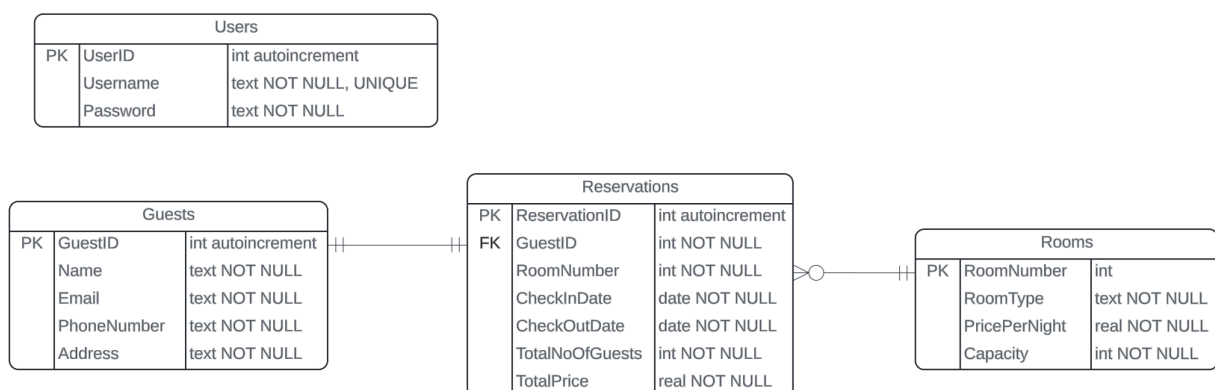


Figure 2 - ERD diagram of reservation system SQL database.

Software Features and Functionalities

The software follows a three-layer architecture, consisting of a presentation layer, a business logic layer, and a data access layer. It was developed using Visual Studio 2022 Community Edition and the C# programming language.

3.1. Presentation Layer (UI Layer)

The presentation layer was implemented using Windows Form App (.NET Framework version 4.7.2).

Figure 3 shows the login window of the system. It contains text boxes for entering the user's username and password, a checkbox to show or hide the password, and buttons to log in or exit the application. When the user clicks one of these buttons, an event handler is triggered to call and execute the corresponding method in the Business Logic Layer.

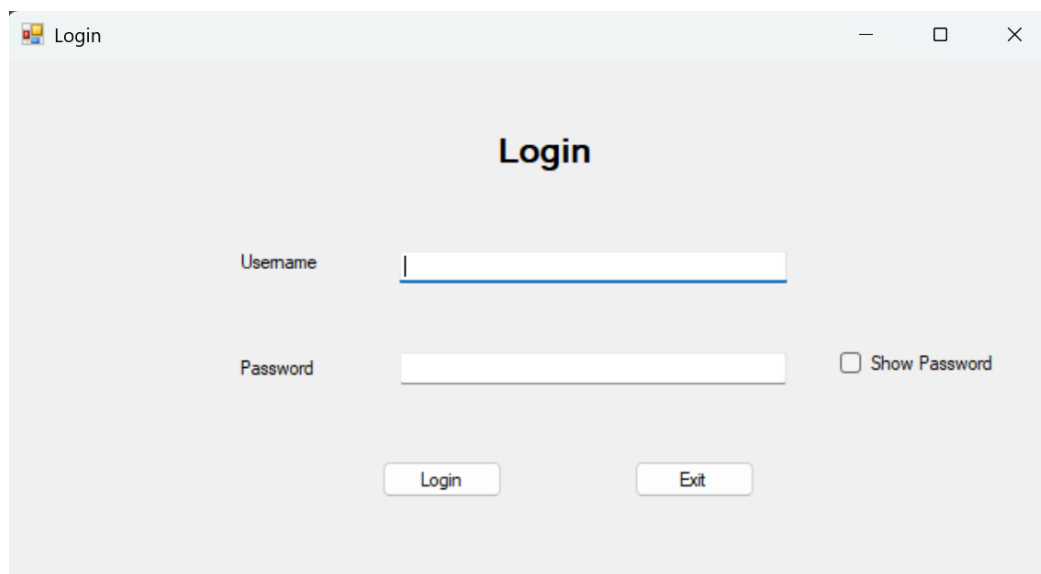


Figure 3 - Login Window.

Figure 4 shows the main window of the hotel reservation system.

The left-side section contains text boxes for entering guest information, radio buttons to select the number of guests, a DateTimePicker for selecting check-in and check-out dates, a DomainUpDown control to choose the room type (single, double, triple, or family), a list box displaying available rooms, and a text box showing the total price. It also includes a button to create a reservation and another to clear the form. The large list box at the bottom displays the details of the created reservation.

The right-side section contains a text box to search for a reservation by ID, a button to perform the search, and a button to clear the search. When a reservation is searched, the form is

automatically filled with the reservation details, and the list box displays the corresponding reservation.

Buttons are provided to update reservation and guest details or to delete a reservation. To perform these operations, a reservation must first be selected from the list box.

Additionally, there is a button to list all reservations ordered by ID and two buttons to sort reservations by guest name or check-in date. All retrieved reservations are displayed in the list box.

Finally, there is a button to close the system.

Hotel Reservation System

Guest Name

Email

Phone Number

Address

Total number of guests ☒ 1 ☐ 2 ☐ 3 ☐ 4

Check-in Date

Check-out Date

Select room type

Total Rate

Enter Reservation Number

Figure 4 – Main window from the Hotel Management System.

3.2. Business Logic Layer (BLL)

3.2.1. Classes

According to the UML diagram presented in the [Section 2](#), the classes shown in the tables below were created.

Guest.cs – public class		
Properties		Parameterized Constructor
public int	Name	-
public string	PhoneNumber	name
public string	Email	email
public string	PhoneNumber	phoneNumber
public string	Address	address
Reservation.cs – public class		
Properties		Parameterized Constructor
public int	ReservationID	-
public int	GuestId	guest.GuestID
public int	RoomNumber	roomNumber
public DateTime	CheckInDate	checkInDate
public DateTime	CheckOutDate	checkOutDate
public int	TotalNoOfGuests	totalNoOfGuests
public decimal	TotalPrice	totalPrice
public Guest	Guest	guest
Room.cs – abstract public class (parent class)		
Properties		Parameterized Constructor
public int	RoomNumber	roomNumber
public decimal	PricePerNight	pricePerNight
public int	Capacity	capacity
SingleRoom.cs – public class (child class from Room)		
Properties		Derived Class Constructor
Same as the parent class Rooms		Calls constructor from the base class Room
DoubleRoom.cs – public class (child class from Room)		
Properties		Derived Class Constructor
Same as the parent class Rooms		Calls constructor from the base class Room
TripleRoom.cs – public class (child class from Room)		
Properties		Derived Class Constructor

Same as the parent class Rooms	Calls constructor from the base class Room
FamilyRoom.cs – public class (child class from Room)	
Properties	Derived Class Constructor
Same as the parent class Rooms	Calls constructor from the base class Room

3.2.2. Object oriented programming concepts

➤ Encapsulation

The classes were created with properties and parameterized constructors to allow the creation, reading, and modification of objects of the class.

➤ Abstraction

The **Room** class is abstract, preventing the creation of a generic room object without specifying a type (single, double, triple or family).

➤ Inheritance

The **Room** class is a parent class, and **SingleRoom**, **DoubleRoom**, **TripleRoom** and **FamilyRoom** are child classes. They inherit the properties of the parent class, which helps avoid repetition in the code.

3.2.3. Form validation rules

Guest's Info	Name	Email	Phone Number	Address
Rules	At least first and last name with minimum of 3 letters each.	Must contain characters before and after the @ symbol, a valid domain, and at least 2 characters after the dot.	Must include a country code (1–3 digits) followed by a space and the phone number (8–15 digits).	Cannot be empty and may contain letters, numbers, spaces, commas, and hyphens.

Reservation's Details	Capacity Room	Check-in Check-out Date	Room Type and Number	Total Price
Rules	Cannot exceed the room's maximum capacity.	1) Check-out date must be after check-in date. 2) New reservations must be for the present or future; past dates are not allowed.	A room must be selected.	Must be a valid positive decimal number.

3.2.4. Services: Sorting and Searching

Four classes were implemented in the service layer to perform sorting and searching operations on reservation objects.

SortReservationsById.cs

The class contains the ***InsertionSortByReservationID()*** method, which takes a list of reservation objects as a parameter and performs an insertion sort algorithm based on the ReservationID property. In this particular case, the insertion sort has a time complexity of $O(n)$ because the list is already sorted by the database, as the ***ReservationID*** field is an auto-incrementing integer.

SortReservationsByName.cs

The class contains the ***BubbleSortByGuestName()*** method, which takes a list of reservation objects as a parameter and performs a bubble sort algorithm based on the guests' ***Name*** property. It sorts the list alphabetically from A to Z and has a time complexity of $O(n^2)$.

SortReservationsByDate.cs

The class contains the ***MergeSortByCheckInDate()*** method, which takes a list of reservation objects as a parameter and performs a merge sort algorithm based on the reservation ***CheckInDate*** property. It sorts the list chronologically and has a time complexity of $O(n \log n)$.

BinarySearchById.cs

The class contains the ***BinarySearch()*** method, which takes a sorted list of reservation objects and a reservation ID as parameters. It performs a binary search algorithm to return the reservation object matching the input reservation ID. If no reservation is found, it returns *null*.

3.2.5. Methods and Logic

Form2 (Login Window)

- `loginButton_Click` – private event handler method
 1. Retrieves the username and password entered by the user.
 2. Validates credentials by calling `ValidateUser()` method from the Database Access Layer (DAL), passing the username and password as parameters.
 3. Returns a success or failure message to user.
 4. If the validation is successful, opens Form1, which is the main Hotel Reservation System window.
 5. If the validation fails, remains on the login window, allowing the user to try different credentials.
- `checkBoxShowPassword_CheckedChanged` – private event handler method
 1. Toggles the visibility of the password characters in the password text box.
- `exitButton_Click` – private event handler method
 1. Closes the application.

Form1 (Hotel Reservation System Window)

- `ValidateForm` – private method
 1. Returns *false* if any validation step fails (steps 2–9 below).
 2. Validates the guest's name using a REGEX pattern according to the rules described in the previous section. If the input is invalid, an error icon and message are displayed.
 3. Applies the same logic to validate email, phone number, and address, each using its respective REGEX pattern.
 4. Checks whether a room type has been selected by the user.
 5. Checks whether a room number has been selected by the user.
 6. Ensures that the total number of guests does not exceed the selected room's maximum capacity. If the capacity is exceeded, a warning icon and message are displayed.
 7. Ensures the check-in date occurs before the check-out date by comparing the two dates.
 8. Prevents creating reservations with past dates by comparing the check-in date with the current date.
 9. Ensures the total rate is a positive decimal number by attempting to parse the input string to a decimal and verifying that it is greater than 0.

10. Returns *true* if all validations are successful.

- **UpdateRoomNumber** – private method

1. Checks whether a room type has been selected.
2. Retrieves the selected room type.
3. Retrieves the check-in and check-out dates.
4. Clears the rooms ListBox.
5. Retrieves available rooms from the database by calling the ***GetAvailableRoomsByTypeAndDate()*** method, passing the room type, check-in, and check-out dates as parameters.
6. Displays the available rooms in the ListBox.
7. Selects the first available room; if no rooms are available, displays a message to the user.

- **roomType_SelectedItemChanged** – private event handler method

1. Calls the ***UpdateRoomNumbers()*** method when a room type is selected, updating the available rooms in the ListBox instantly.

- **checkin_ValueChanges** – private event handler method

1. Calls the ***UpdateRoomNumbers()*** and ***UpdateTotalPrice()*** methods when the check-in date changes, updating the available rooms and total price instantly.

- **checkout_ValueChanges** – private event handler method

1. Calls the ***UpdateRoomNumbers()*** and ***UpdateTotalPrice()*** methods when the check-out date changes, updating the available rooms and total price instantly.

- **UpdateTotalPrice** – private method

1. Checks whether a room number is selected; if no room is selected, displays an empty string in the total rate text box.
2. Retrieves the room object; if the object is *null*, displays an empty string in the total rate TextBox.
3. Retrieves the check-in and check-out dates and checks whether the check-out date is after the check-in date; if not, displays an empty string in the total rate TextBox.

4. Calculates the number of nights by taking the difference between the check-out and check-in dates.
5. Calculates the total price by multiplying the price per night of the selected room by the total number of nights.
6. Displays the total price in the total rate TextBox.

- **SelectRoomNo_SelectedIndexChanged – private method**

1. Calls the **UpdateTotalPrice()** method when a different room is selected in the list box, updating the total price instantly.

- **createReservationButton_Click – private event handler method**

1. Performs input data validation by calling the **ValidadeForm()** method.
2. Retrieves the total number of guests selected from RadioButtons.
3. Retrieves the check-in and check-out dates from the DateTimePicker.
4. Retrieves the room type and number from the selected room object in the ListBox.
5. Checks whether the total number of guests exceeds the selected room's capacity.
6. Calculates the total number of nights by taking the difference between the check-out and check-in dates.
7. Calculates the total price by multiplying the room's price per night by the total number of nights.
8. Retrieves the guest's information from the form.
9. Creates new **Guest** object by passing guest information parameters.
10. Adds new guest to the database by calling the **AddGuest()** method from the DAL and retrieves the assigned **GuestID**.
11. Creates a new **Reservation** object by passing reservation details parameters.
12. Adds new reservation to the database by calling the **AddReservation()** method from the DAL.
13. Displays a message indicating that the reservation was created successfully.
14. Clears ListBox and displays new reservation created.

- **updateReservationButton_Click – private event handler method**

1. Checks whether a reservation is selected in the list box. If no reservation is selected, prompts the user to select one.
2. Performs input data validation by calling the **ValidadeForm()** method.
3. Retrieves the selected reservation from the list box.
4. Updates the guest object details (name, email, phone number and address).
5. Updates guest's record in the database by calling the **UpdateGuest()** method.
6. Retrieves the total number of guests selected from RadioButtons.

7. Retrieves the room type and number from the selected room object in ListBox.
8. Checks whether the total number of guests exceeds the selected room's capacity.
9. Updates the reservation object details (check-in, check-out, room number, total number of guests, and total rate).
10. Updates reservation record in the database by calling **UpdateReservation()** method from the DAL.
11. Displays a message indicating that the reservation was updated successfully.
12. Clears the list box and displays the updated reservation.

- **deleteReservationButton_Click** – private event handler method

1. Checks whether a reservation is selected in the list box. If no reservation is selected, prompts the user to select one.
2. Retrieves the selected reservation from ListBox.
3. Prompts the user to confirm the deletion of the reservation; the user must select 'Yes' or 'No'.
4. If user selects 'Yes', call the **DeleteReservation()** method from DAL, passing the ID of the selected reservation.
5. Removes reservation from ListBox.
6. Displays a message indicating that the reservation was deleted successfully.

- **searchReservationButton_Click** – private event handler method

1. Validates the user input by attempting to parse the input string as an integer and verifying that it is greater than 0.
2. Retrieves all reservations and stores them in a list by calling the **GetAllReservations()** method from DAL.
3. Sorts all reservations by ID in ascending order by calling the **InsertionSortByReservationID()** method from the service layer.
4. Performs binary search by calling **BinarySearch()** method to find the reservation with the input ID.
5. If the reservation is found, displays the reservation information in ListBox and calls the **LoadReservationIntoForm()** method to autofill the UI form with the guest's information and reservation details.
6. If no reservation is found, displays a message to the user.

- **listAllReservationsButton_Click** – private event handler method

1. Retrieves all reservations and stores them in a list by calling the ***GetAllReservations()*** method from DAL.
2. If no reservations are found, displays a message to the user.
3. Displays all reservations in ListBox using a foreach loop.
4. Displays a message indicating that the reservations were loaded successfully.

- **sortByNameButton_Click** – private event handler method

1. Retrieves all reservations and stores them in a list by calling the ***GetAllReservations()*** method from DAL.
2. If no reservations are found, displays a message to the user.
3. Calls the ***BubbleSortByGuestName()*** method from the service layer to sort the reservations in the list alphabetically by guest names (A–Z).
4. Clears ListBox and displays all reservations using a foreach loop.
5. Displays a message indicating that the reservations were sorted successfully.

- **sortByCheckInButton_Click** – private event handler method

1. Retrieves all reservations and stores them in a list by calling the ***GetAllReservations()*** method from DAL.
2. If no reservations are found, displays a message to the user.
3. Calls the ***MergeSortByCheckInDate()*** method from the service layer to sort the reservations in the list by check-in date, from earliest to latest.
4. Clears ListBox and displays all reservations using a foreach loop.
5. Displays a message indicating that the reservations were sorted successfully.

- **LoadReservationIntoForm** – private method

1. Takes ***Reservation*** object as a parameter.
2. Sets flag to prevent ***UpdateRoomNumbers()*** method from calling ***GetAvailableRoomsByTypeAndDate()*** and indicate that the room is not available.
3. Checks whether the reservation object or its ***Guest*** property is *null*.
4. Fills the guest's information into the corresponding TextBoxes in the form.
5. Fills reservations details to their respective Windows Form controls (details provided in the [UI layer section](#)).

- **listBox1_SelectedIndexChanged** – private event handler method
 1. Checks whether a reservation is selected in the list box; if no reservation is selected, the method returns.
 2. If a reservation is selected, calls the ***LoadReservationIntoForm()*** method to populate the form with the corresponding guest and reservation details.

- **ClearAllInputForm** – private method
 1. Clears all input fields in the form.

- **clearForm_Click** – private event handler method
 1. Calls the ***ClearAllInputForm()*** method when the “Clear Form” button is clicked.

- **clearSearchButton_Click** – private event handler method
 1. Calls the ***ClearAllInputForm()*** method to clear the form and clears the last input reservation ID from search TextBox when the “Clear Search” button is clicked.

- **Form1_FormClosed** – private event handler method
 1. Exits the application when the form is closed using the ‘X’ button in the top-right corner.

- **closeButton_Click** – private event handler method
 1. Exits the application when the “Close System” button is clicked.

4.3. Data Access Layer (DAL)

The following table summarizes the CRUD operations performed by the Data Access Layer (DAL).

Create	Read	Update	Delete
Guests	Guest's info	Guest's info	Reservations
Reservations	Reservations' details	Reservations' details	
Rooms	Rooms' details		
Users	Users' credentials		

4.3.1. Methods and Logic

- **Initialize** – public static method

The method initializes the database and creates all tables if they do not already exist, according to the Entity Relationship Diagram presented in [Section 3](#).

After, it populates the **Rooms** table with predefined room numbers, types, prices per night, and capacities. For example:

```
INSERT OR IGNORE INTO Rooms (RoomNumber, RoomType, PricePerNight, Capacity)
VALUES (101, 'Single Room', 100, 1).
```

Lastly, it creates a default user credential in the **Users** table if it does not already exist, using the **INSERT OR IGNORE** command.

- ❖ Username: admin
- ❖ Password: password

- **AddGuest** – public static method

The method creates a guest record in the **Guest** table using the information stored in the **Guest** object and returns the assigned **GuestID**.

- **GetAvailableRoomsByTypeAndDate** – public static method

The method first selects the available rooms for the specified period between the check-in and check-out dates. It works by applying a condition that excludes all rooms of the selected type (single, double, triple, or family) that appear in the **Reservations** table and have a conflicting check-in and check-out period.

After, it maps the retrieved room records to **Room** objects and returns a list of the available rooms.

- **AddReservation** – public static method

It creates a reservation record in the **Reservation** table from the information stored in the **Reservation** object.

- **GetAllReservations** – public static method

The method first selects the reservations and respective guest details. After, it maps the reservation records to **Reservation** objects, creates, and returns a list with all reservations.

- **UpdateGuest** – public static method

It updates guest's records based on details from the **Guest** object parameter and returns a message of success or failure if no record is found.

- **UpdateReservation** – public static method

It updates reservations' records based on details from the **Reservation** object parameter and returns a message of success or failure if no record is found.

- **DeleteReservation** – public static method

It deletes a reservation record from the input reservation ID and returns a message of success or failure if no record is found.

- **ValidateUser** – public static method

It validates the user credentials based on the username and password entered in the login window by comparing them with the records in the Users table and returns *true* if a matching record is found.

5. Future Development

Future improvements include adding the ability to search for reservations by guest name, which is a usual feature found in professional hotel management systems.

Another functionality to be implemented is allowing a guest to have multiple reservations. Currently, if a guest requires an additional room, a new reservation form must be completed, resulting in the creation of a duplicate record for the same guest in the **Guests** table. Implementing this functionality would optimize the guest table by avoiding duplication of guest information.

At present, each reservation is associated with one and only one room; however, this is also a limitation of the system. Supporting reservations with multiple rooms would eliminate the need to create multiple reservations for the same guest.

Hotels typically retain data for cancelled reservations in order to access it if needed and to analyse performance and customer behaviour. Adding a feature to cancel reservations and store them in a dedicated database table would be a logical next step in the system's development.

Another common feature in hotel management systems is the ability to store guest payment details, such as debit and credit card information. Implementing this functionality would require appropriate data security measures.

The system's design and user interface experience represent another area that requires further development and improvement, particularly to ensure clear, fast, and user-friendly interaction with the system.

References

GeeksforGeeks (n.d.) Unified Modeling Language (UML) | Class Diagrams. Available at: <https://www.geeksforgeeks.org/system-design/unified-modeling-language-uml-class-diagrams/> (Accessed: 12 November 2025).

IronPDF (2025) C# SQLite guide. Available at: <https://ironpdf.com/blog/net-help/csharp-sqlite-guide/> (Accessed: 21 November 2025).

Lucidchart (n.d.) UML class diagram tutorial. Available at: <https://www.lucidchart.com/pages/uml-class-diagram> (Accessed: 12 November 2025).

Lucidchart (n.d.) *What is an Entity Relationship Diagram (ERD)?* Available at: <https://www.lucidchart.com/pages/er-diagrams> (Accessed: 20 November 2025).

Manzar, K.A. (2025) 3-layer Architecture. [PowerPoint presentation] Department of Computer Science, Dublin Business School.

Manzar, K.A. (2025) Advanced Programming. Available at: <https://github.com/KAAlidBS/Advanced-Programming/tree/main#> (Accessed: 1 November 2025).

Manzar, K.A. (2025) Binary Search: PhoneBook with User Interface. [PowerPoint presentation] Department of Computer Science, Dublin Business School.

Manzar, K.A. (2025) OOP Concepts. [PowerPoint presentation] Department of Computer Science, Dublin Business School.

Manzar, K.A. (2025) Sorting Selection Sort, Bubble, Insertion, Merge. [PowerPoint presentation] Department of Computer Science, Dublin Business School.

Microsoft (n.d.) How to subscribe to and unsubscribe from events. Available at: <https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/events/how-to-subscribe-to-and-unsubscribe-from-events> (Accessed: 30 November 2025).

Microsoft (n.d.) Object-oriented programming (C#). Available at: <https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/tutorials/oop> (Accessed: 10 November 2025).

Microsoft (n.d.) Inheritance in C#. Available at: <https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/tutorials/inheritance> (Accessed: 1 November 2025).

Microsoft (n.d.) Using SQLite in .NET. Available at: <https://learn.microsoft.com/en-us/dotnet/standard/data/sqlite/?tabs=net-cli> (Accessed: 22 November 2025).

Stack Exchange (n.d.) Constraint to prevent overlapping time periods. Available at: <https://dba.stackexchange.com/questions/133036/constraint-to-prevent-overlapping-time-periods> (Accessed: 3 December 2025).

W3Schools (n.d.) SQL Data Types. Available at: https://www.w3schools.com/sql/sql_datatypes.asp (Accessed: 15 November 2025).

YouTube (2020) How to Create Login Form in C# 2020. Available at: <https://www.youtube.com/watch?v=n2FWwmgMKok> (Accessed: 1 December 2025).

YouTube (2024) SQLite Database Installation on Visual Studio 2022 | 2024. Available at: <https://www.youtube.com/watch?v=7TpljNN0IvA> (Accessed: 23 November 2025).